

Machine Learning Architectural Decision Record and Final Report

Ernesto Ordonez, Jose Diaz, Brian Rewko, Sung Yoon, Majd R

Requirements:

Python Version: 3.11.14

What are ADR's? A rationale and justification of every significant decision in an engineering project. Data science and Machine learning can be used to identify and analyze potential consequences, advantages, and disadvantages.

1. Executive Summary and Project Goals

The core purpose of this project is to develop a machine learning-driven system for the detection and prevention of Denial of Service (DoS) attacks to strengthen organizational network security. DoS attacks are cyber assaults specifically designed to render networks, services, and devices unavailable for use by legitimate users. This is achieved through overwhelming the system by flooding it with an enormous volume of artificial traffic which strains resources and pushes the system beyond its capacity. Attacks also function through exploiting specific system vulnerabilities to induce a crash. The goal of this project is to create an Intrusion Detection and Prevention System (IDS/IPS) that uses a predictive model to identify when a network is being targeted for such an attack. This addresses the critical need for Business to function smoothly, as failure to mitigate a DoS attack results in costly service downtime and reputational damage making rapid, automated detection an absolute priority.

The project's analytical foundation is built upon the CIC-IDS2017 dataset, a modern and comprehensive resource widely acknowledged in the security research community for its realism and completeness. CIC-IDS2017 was engineered to mimic a true, real-world network environment, capturing a detailed profile of human interactions across common protocols like HTTP, HTTPS, FTP, SSH, and email over five consecutive days of network activity. This collection of data includes the entirety of the network exchange in large PCAP (packet capture) files, which contain every individual packet and its payload. The traffic is diverse, encompassing typical daily benign traffic alongside a spectrum of simulated, contemporary attacks.

The raw PCAP files were analyzed by the CICFlowMeter tool, which converts the continuous stream of individual packets into discrete network flows. A flow represents a complete session or conversation, uniquely identified by a combination of key attributes: the source and destination Internet Protocol (IP) addresses, the source and destination port numbers, the transport protocol (TCP or UDP), and a timestamp.

The core of the dataset, and the direct input for the machine learning algorithms, consists of 78 distinct, numerical features that the CICFlowMeter tool extracts for every recorded flow. These features provide a numerical summary of the network session. They quantify metrics essential for pattern recognition, such as statistical summaries of packet lengths (minimum, maximum, mean, and standard deviation), the total number of packets sent in each direction, and crucial timing data, including the total

flow duration and the mean inter-arrival time between packets. By quantifying these characteristics, the 78 features enable the classifier to find differences in traffic.

The data specifically focused on the traffic captured on Wednesday, as it contains both benign traffic and the core DoS threats for the classification task. This selection process highlights the diversity within the DoS attack vector itself. The dataset includes traffic from four distinct DoS variants, illustrating the crucial difference between high-volume and stealthy attacks: DoS Hulk, which is a high-volume DoS attack where the attacker generates a huge volume of HTTP requests to quickly consume all of the server's resources (an easily detectable inlier); and DoS GoldenEye (along with related attacks DoS slowhttptest and DoS slowloris), which is a type of application-layer DoS attack that aims to keep connections alive for a long time by sending slow, randomized HTTP requests using cache control and keep-alive headers. The slow and persistent nature of DoS GoldenEye makes it an outlier that is inherently more difficult to detect than a volumetric attack.

Following the initial data isolation, two final steps were required to prepare the data for the supervised learning task: first, removing flows associated with the Heartbleed attack, which, as a vulnerability exploitation, falls outside the scope of DoS denial-of-service attempts; and second, grouping the remaining four distinct DoS attack labels (GoldenEye, Hulk, slowhttptest, slowloris) into a single, unified class labeled 'Attack'. This creates the perfect supervised learning setup, where the machine learning model's ultimate mission is to look at the 78 flow features and output one of two predictions: 'Benign' or 'Attack'. This simplification is critical for developing a high-speed, high-performance frontline security tool where the instantaneous decision to block or permit the flow is paramount, regardless of the attack's specific name. The Technical Goals center on achieving near-perfect accuracy and minimal false alarms to ensure a reliable defense, directly translating to the Operational Goal of preventing service disruption and protecting the business from financial and reputational harm.

2. Data Sources

2.1 context

For the development of the intrusion detection system (IDS), the project requires a dataset which provides a realistic, diverse and fully labelled malicious network traffic. The goal is to train a machine learning model to accurately perform binary classification that must detect both high volume denial of service attacks and more subtle protocol level exploits. To ensure the reproducibility, the dataset must include

- Ground truth labels for benign and dos attacks
- Multiple attack types executed
- Realistic background traffic representative of normal user behavior

Most publicly available IDS datasets fail to meet the requirements due to limitations such as outdated traffic, incomplete labeling, and narrow attack diversity. This makes it unreliable as a training dataset.

2.2 Justification

This project uses the CIC-IDS2017 dataset, with a focus on the Wednesday attack scenario. The Wednesday portion contains a structured sequence of DoS attacks. These attacks were executed in real network environments with a complete firewall chain for an accurate modeling of both attacker behavior and packet routing conditions.

3. Data Integration

3.1 Context

The project relies exclusively on the CIC-IDS2017 dataset for the network intrusion data. Unlike typical machine learning systems that draw information from different sources this project aims at a single well structured dataset. The Wednesday subset which includes DoS, DDoS, and Heartbleed attacks which provides all the necessary raw data in CSV, Json, and Parquet formats.

Because there is no external sources the extraction focuses on the following

- Identifying data files
- Loading files into dataframes
- Maintaining original dataset labeling and data integrity for later stages

3.2 Justifications

The decision to use only the CIC-IDS2017 dataset and to stage all data in pandas is justified through the following

- Using a single dataset allows us to be consistent and prevents inconsistencies which could arise from different datasets
- By using pandas we can efficiently provide an efficient in memory staging process where transformations, cleaning steps, and exploratory analysis can be performed without additional data structures

3.3 Status

Accepted (Jose) 11/15/25 after confirming that all required attack data for model training and evaluation is contained within the Wednesday dataset, eliminates the need for additional data sources.

4. Data Transformation

4.1 Context and Technology Choice

Understanding the methods of explorations being done based on the complexity of our dataset. The dataset contains 54,593 samples and 79 features, making it moderately large and possibly diverse in data types. Because of this complexity, Python with Pandas and NumPy is chosen for exploration due to their flexibility, speed and ability to handle mixed data types. These tools allow us to inspect column types and values, detect inconsistencies, clean naming issues, and evaluate missing values efficiently before moving onto feature engineering or modeling.

4.2 Justifications

Pandas is well-suited for this dataset because it handles large tabular data efficiently and provides built-in tools for type **inspection**, **cleaning**, and **transformations**. Using Pandas allows us to

- quickly identify the distribution of datatypes (inter, float, string),
- fix formatting issues (such as leading spaces in column names),
- quantify missing or corrupted values,
- quickly compute summary statistics

4.3 Status

Ernesto (Accepted) :The dataset has been successfully loaded into a Pandas DataFrame. Initial exploratory steps were conducted, including:

Displaying full list of columns and the dataset shape(54,593 samples, 79 features),

Counting DataTypes: **54 Integer, 24 float, 1 String column,**

Detecting improper formatting, including leading spaces in column names, which were removed to prevent naming or processing errors,

Identifying missing values, with 150NaN values found across the entire DataFrame.

Further inspection revealed that the single **String**-based feature contains three **unique** values: **BENIGN, DoS Hulk, DoS GoldenEye**. This column is recognized as the datasets **label(target variable)**. The remaining 78 **columns**(features) represent numerical features. From this, we conclude that our project follows a **Supervised Learning Framework**, since we have clearly defined **target column** is used to classify each sample.

5. Data Storage (Load)

5.1 Context and Technology Choice

Once all transformations and preprocessing steps are completed the dataset is formatted for continued experimentation, reproducibility, and efficient reuse by machine learning engineers. In this project, the transformed data is loaded and processed into Pandas Dataframes not needing the infrastructure for larger scale data storage.

The data is static and moderate size and used in an iterative research environment where rapid experimentation and ease of access are more important than having a distributed storage so we store the data as a CSV. this format is lightweight and human readable with easily accessible tools.

5.2 Justifications

The decision to use Pandas Dataframes to stage

- Seamless integration with the Machine Learning workflow (scikit learn, numpy).
- Eliminate the need for unnecessary database queries before modeling
- Performance is optimized with vectorized operations, fast filtering, and efficient column selection for feature engineering.

The decision to use CSV to store data

- CSV is compatible with pandas, scikit learn, numpy, and most machine learning frameworks
- Avoid repetitive extraction
- Reproducibility by storing consistent and preprocessed versions of the dataset

5.3 Status

Jose Diaz (Accepted): Chosen after determining that a preprocessed, single file csv provides the fastest and most reproducible workflow for subsequent machine learning experiments without requiring a database

6. Reading Data

6.1 Context

After completing the ETL load stage, the transformed dataset is read into pandas Dataframes for all tasks including data analysis, feature engineering, model development, and evaluation.

The Pandas Dataframes offer seamless integration with the machine learning frameworks and standardization tools. This enables efficient manipulation, slicing, filtering, and merging of the datasets.

6.2 Justification

The data was stored in a Pandas dataframe which seamlessly fits into the Machine Learning workflow as well as the following.

- Native support for csv and tubular data
- Alignment with existing project workflow
- Compatibility with machine learning libraries

6.3 Status

Accepted (Brian, Jose, Majd, Ernesto,Sung) The dataset of the project has already been stored in a pandas dataframe which allows us to execute all of the machine learning tasks efficiently.

7. Exploratory Data Analysis

7.1 Context

Exploratory Data Analysis is essential for understanding the dataset and making informed decisions about model selection and feature engineering. EDA includes the following.

- Identifying the shape of the dataset
- Unify the column / feature name
- Identifying the unique variables for class labels
- Identifying if the dataset has missing data
- identify high percentage of missing data
- Performing univariate analysis
 - Statistics of columns mean, standard, or variance. Identify columns with low or near zero variance.
 - Visualization
- Generating bivariate analysis
 - Computing and visualizing correlation matrix of datasets

7.2 Justification

The choice made during the EDA process are justified for the following reasons

- Understanding dataset integrity
- Missing value management
- Ensuring numeric compatibility
- Feature reduction via variance analysis
- Label verification
- Statistical Summaries for insight

7.3 Status

Brian (Rejected) 11/24/25: Attempted running a liner model on the dataset but required intensive resources that spiked cpu and ram usage to 99% on a high end machine.

Team (Accepted): EDA methodology approved after successfully validating dataset integrity, removing unusable features and confirming that the data is ready for preprocessing and machine learning models

Machine Learning Pipeline

8. Data Processing

8.1 Context and Technology Choice

Data processing ensures the dataset is properly cleaned and prepared for the machine learning model. Because this dataset contains numerical flow based features and a categorical variable (destination port), preprocessing plays a crucial role in improving the performance and preventing data leakage in the following ways

- Removing duplicates
- Removing missing values
- Encoding categorical data
- Splitting the dataset into testing and training data

8.2 Justification

Based on the datawrangling extension there were no duplicate records so further action was unnecessary.

8.2 Justifications

- Categorical data was not required due to removal of destination port
- The destination port was removed due to no information gained.

8.3 Status

Jose (Accepted): The removal of destination port due to data leakage

Brian (Accepted) 12/4/25, The addition of recursive feature reduction via XGBClassifier

9. Feature Engineering

9.1 Context and Technology Choice

Feature engineering focuses on improving model performance by selecting, refining, or transforming the features that best represent the underlying patterns in the dataset. In our context of Intrusion Network Detection,

We removed data that had lower than or equal to 1 variance.

Applied a standard scalar to standardize the features (mean = 0, std=1)

We removed destination port due to possible data leakage.

Performed recursive backward feature selection from XGboost classifiers.

9.2 Justifications

Wanted to attempt feature reduction through recursive feature reduction to allow the model to decide. After the model has chosen a selected feature, values were modified.

9.3 Status

Accepted (Jose) 11/21/25: Manual feature selection was used

Accepted (Brian, Ernesto) 12/4/25: Applies Recursive Feature Reduction through XGBoost classifier but was cautious on the results due to suspiciously good results with low number of features.

10. Process Data Loading

10.1 Context and Technology Choice

Data was manipulated through the pandas dataframe that contains the filtered data.

10.2 Justifications

Pandas Dataframes are useful with the way their data is structured to allow for easy manipulation of data

10.3 Status

Team (Accepted)

11. Model Selection and Training

11.1 Context and Technology Choice

We decided on the XGboost model and trained it on each iteration

11.2 Justifications

Used XGboost due to the decision tree classifier allowing binary classification and the availability of feature_importances to assist in feature selection

11.3 Status

Accepted (Jose) Using the XGBoost Model for our dataset

12. Model Evaluation

12.1 Context and Technology Choice

Used Classification evaluation metrics to identify how well our model was identifying attack vs benign traffic

12.2 Justifications

Used Classification evaluation due to data consisting of a single label that is binary that identifies attacks as true and benign as false.

12.3 Status

Accepted (Jose) 11/22/25: Used Classification evaluation of Accuracy, Precision, F1-Score, and Recall

14. Concluding Remarks and Project Summary

Throughout the project, we designed and implemented a complete machine learning model for detecting denial-of-service (DoS) attacks using the CIC-ID2017 Wednesday dataset. The project began with establishing a clear understanding of the dataset structure, identifying 78 numerical features and a single categorical label column containing three classes: BENIGN, DoS Hulk, DoS GoldenEye. After consolidating the three minority/malicious classes into a single 'attack' label, the problem was formally framed as a binary supervised learning task: distinguishing legitimate traffic from attack traffic.